



Herança

O que é Herança?

- Herança é um dos pilares da Programação Orientada a Objetos (POO), ao lado de encapsulamento, abstração e polimorfismo.
- Permite que uma nova classe (subclasse) **reutilize** e **especialize** os comportamentos de uma classe existente (superclasse).
- Objetivos principais:
 - **Reutilização de código** já testado e confiável.
 - **Especialização de comportamentos** para classes mais específicas.

Vantagens e Relações “é um” vs. “tem um”

- Vantagens da herança:

- Evita repetição de código.
- Centraliza manutenção de funcionalidades comuns.
- Facilita a extensão e escalabilidade do sistema.

- Relações conceituais:

- “É um” → Herança (ex: Carro é um tipo de Veículo)
- “Tem um” → Composição ou agregação (ex: Carro tem um Motor)

Exemplo introdutório: Veículo e Carro

// Superclasse

```
public class Veiculo {
    private String marca;

    public Veiculo(String marca) {
        this.marca = marca;
    }

    public void exibirMarca() {
        System.out.println("Marca: " + marca);
    }
}
```

// Subclasse

```
public class Carro extends Veiculo {
    private String modelo;

    public Carro(String marca, String modelo) {
        super(marca); // chama o construtor da superclasse
        this.modelo = modelo;
    }

    public void exibirModelo() {
        System.out.println("Modelo: " + modelo);
    }
}
```

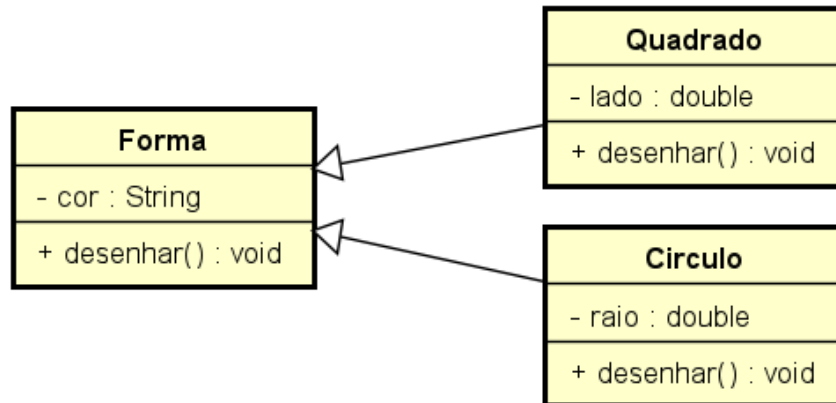


Superclasse e Subclasse: Definições

- Uma **superclasse** (ou classe base, ou classe-pai, ou classe-mãe) é aquela que fornece atributos e métodos comuns a várias outras classes.
- Uma **subclasse** (ou classe derivada, classe-filha) herda os atributos e métodos da superclasse e pode:
 - Adicionar novos comportamentos;
 - Modificar comportamentos herdados;
 - Especializar funcionalidades.
- Relação:
 - Subclasse é um tipo **mais específico** da Superclasse.

Hierarquias de Classes

- Hierarquias representam níveis de **especialização**:
 - Exemplo: Forma → Circulo, Quadrado
- Permitem organizar o sistema de forma mais **compreensível** e **modular**.
- As subclasses herdam **características comuns** e adicionam **detalhes próprios**.

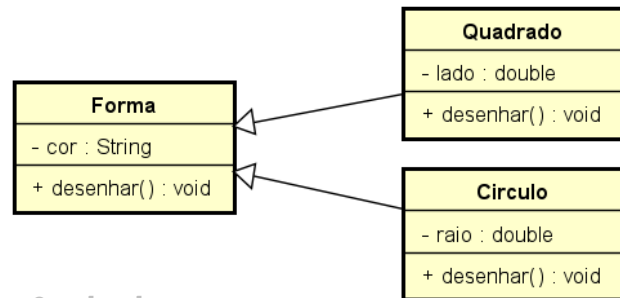


Hierarquias de Classes

- Classe Forma: contém atributo cor e método desenhar().
- Classe Quadrado: herda cor, adiciona lado e implementa area().
- Classe Circulo: herda cor, adiciona raio e implementa area().

```
// Superclasse
public class Forma {
    public void desenhar() {
        System.out.println("Desenhando forma genérica.");
    }
}
```

```
// Subclasse Círculo
public class Circulo extends Forma {
    @Override
    public void desenhar() {
        System.out.println("Desenhando um círculo.");
    }
}
```



```
// Subclasse Quadrado
public class Quadrado extends Forma {
    @Override
    public void desenhar() {
        System.out.println("Desenhando um quadrado.");
    }
}
```

A palavra-chave `extends` em Java

- Em Java, a herança entre classes é declarada com a palavra-chave `extends`.
- Sintaxe básica:

```
class Subclasse extends Superclasse {  
    // atributos e métodos específicos da subclasse  
}
```

- Ao usar `extends`, a subclasse **herda** todos os membros públicos (`public`) e protegidos (`protected`) da superclasse.
- A herança permite **reaproveitamento de código** e **especialização de comportamento**.

Regras sobre extends

- Java suporta apenas **herança simples**:
 - Uma subclasse **pode herdar de apenas uma** superclasse.

```
class Animal {}
```

```
class Cachorro extends Animal {}
```



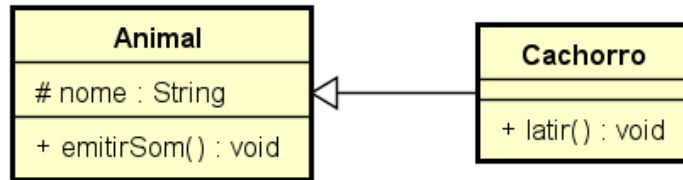
```
class Cachorro extends Mamifero, Carnivoro {}
```



- Toda classe herda, direta ou indiretamente, da classe `Object`.
- Não é necessário escrever `extends Object`, pois isso ocorre **implicitamente**.

O que se herdada em uma subclasse

- Uma subclasse herda da superclasse:
 - Todos os **métodos** declarados como `public` e `protected`.
 - Todos os **atributos** `public` e `protected`.
- Esses membros herdados podem ser utilizados diretamente na subclasse, a menos que sejam sobrescritos ou ocultados.



```

public class Animal {
    protected String nome;

    public void emitirSom() {
        System.out.println("Som genérico");
    }
}
  
```

```

public class Cachorro extends Animal {
    public void latir() {
        System.out.println(nome + " está latindo");
    }
}
  
```

O que não é herdado em uma subclasse

- Não são herdados:
 - Construtores da superclasse.
 - Membros com modificador `private`.
- Isso significa que:
 - A subclasse precisa declarar seus próprios construtores.
 - A subclasse não pode acessar diretamente atributos `private` da superclasse.
- Para acessar membros `private`, utiliza-se getters/setters fornecidos pela superclasse.

Revisão: Modificadores de Acesso

- Em Java, o modificador de acesso determina onde um membro (atributo ou método) pode ser acessado:
 - `private`: acesso apenas dentro da própria classe.
 - `protected`: acesso dentro da classe, subclasses e pacote.
 - `public`: acesso irrestrito de qualquer lugar.
- Em hierarquias de herança, o modificador de acesso impacta diretamente o que é herdado e como é acessado.

Diferenças importantes em Herança

- `protected` permite que subclasses acessem membros diretamente, mesmo que estejam em outro pacote.
- Apesar disso, **é preferível evitar** membros `protected`:
 - Pode quebrar o princípio do encapsulamento.
 - Deixa a implementação mais rígida e sujeita a mudanças acopladas.
- Em vez de usar `protected`, prefira:
 - Manter os atributos como `private`.
 - Criar métodos `public` ou `protected` para acesso controlado.

Boas Práticas em Herança (Engenharia de Software)

- Evite ao máximo atributos `protected`.
- Prefira encapsular atributos com `private` e criar métodos de acesso controlado (`get / set`) quando necessário.
- Permite melhor controle sobre como os dados são acessados ou modificados.
- Torna o código mais modular, seguro e de fácil manutenção.

Construtores em Hierarquia de Herança

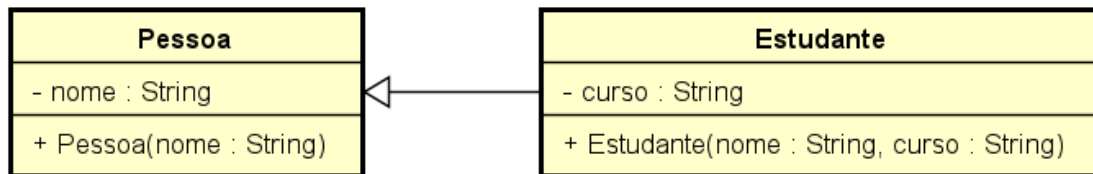
- Construtores não são herdados em Java.
- Toda subclasse deve chamar explicitamente ou implicitamente um construtor da superclasse.
- Para isso, usamos a palavra-chave `super()` dentro do construtor da subclasse.
- Regra obrigatória: a chamada a `super()` deve ser a primeira instrução do construtor da subclasse.

`super()` : Chamada Explícita vs. Implícita

- **Chamada explícita:**
 - Quando usamos `super(parametros)` no início do construtor da subclasse.
- **Chamada implícita:**
 - Se não escrevermos `super()`, o compilador adiciona automaticamente uma chamada ao **construtor sem parâmetros da superclasse**.
- Se a superclasse **não tiver construtor padrão**, devemos obrigatoriamente fazer a chamada explícita.

Exemplo Prático: Construtores

- Vamos observar como a chamada ao construtor da superclasse é feita na subclasse:



```
public class Pessoa {
    private String nome;

    public Pessoa(String nome) {
        this.nome = nome;
    }
}
```

```
public class Estudante extends Pessoa {
    private String curso;

    public Estudante(String nome, String curso) {
        super(nome); // Chamada explícita ao construtor da superclasse
        this.curso = curso;
    }
}
```

Herança Universal em Java: classe `Object`

- Em Java, todas as classes herdam, direta ou indiretamente, da classe `Object`.
- Isso significa que qualquer classe criada por você já possui métodos herdados da classe `Object`, mesmo sem usar `extends`.
- A classe `Object` é definida no pacote `java.lang`, importado automaticamente.

Métodos Herdados da Classe Object

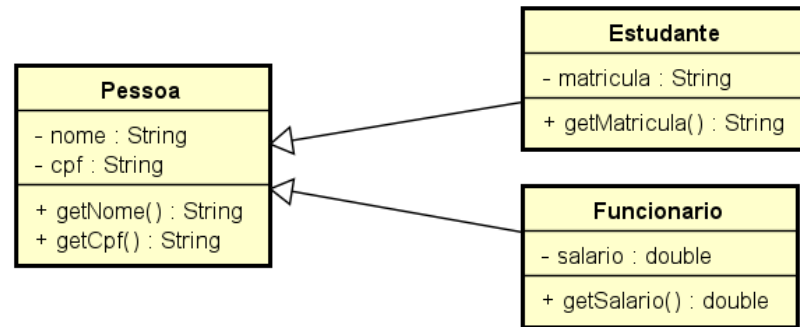
- Os métodos mais comuns herdados de `Object` são:
 - `toString()` – Retorna uma representação em texto do objeto.
 - `equals(Object obj)` – Compara dois objetos para verificar se são “iguais”.
 - `hashCode()` – Retorna um código de hash do objeto, usado em estruturas como `HashMap`.
- Você pode sobrescrever esses métodos para adaptá-los à lógica da sua aplicação.



Veremos como fazer isso no
nosso próximo assunto:
Polimorfismo

Exemplo: Comunidade Acadêmica

- Podemos representar diferentes tipos de pessoas em uma instituição de ensino através de herança.
- A superclasse `Pessoa` representa atributos comuns a indivíduos da instituição.
 - Contém nome e cpf.
- Subclasses especializam `Pessoa`:
 - `Estudante` **adiciona** matricula.
 - `Funcionario` **adiciona** salario.
- Utiliza `super(...)` para inicializar construtores herdados.



Exemplo: Comunidade Acadêmica

```
public class Pessoa {  
    private String nome;  
    private String cpf;  
  
    public Pessoa(String nome, String cpf) {  
        this.nome = nome;  
        this.cpf = cpf;  
    }  
  
    public String getNome() { return nome; }  
    public String getCpf() { return cpf; }  
}
```

Exemplo: Comunidade Acadêmica

```
public class Estudante extends Pessoa {  
    private String matricula;  
  
    public Estudante(String nome, String cpf, String matricula) {  
        super(nome, cpf);  
        this.matricula = matricula;  
    }  
  
    public String getMatricula() { return matricula; }  
}
```

Exemplo: Comunidade Acadêmica

```
public class Funcionario extends Pessoa {  
    private double salario;  
  
    public Funcionario(String nome, String cpf, double salario) {  
        super(nome, cpf);  
        this.salario = salario;  
    }  
  
    public double getSalario() { return salario; }  
}
```

Boas Práticas de Herança (1/4)

- Minimizar o uso de membros `protected`:
 - Evite tornar atributos `protected` indiscriminadamente.
 - Mesmo subclasses não devem acessar diretamente atributos de outra classe; prefira o acesso através de métodos controlados.
 - **Problema**: Acesso direto reduz o encapsulamento e dificulta futuras alterações internas sem afetar subclasses.

Boas Práticas de Herança (2/4)

- Não quebrar o princípio do encapsulamento:
 - Prefira sempre atributos `private` e disponibilize acesso controlado através de **getters** e **setters**.
 - Garanta que modificações internas da superclasse não comprometam o funcionamento das subclasses.
 - Se possível, desenhe classes de forma que alterações internas não “vazem” para o exterior.

Boas Práticas de Herança (3/4)

- Usar corretamente herança e agregação/composição:
 - Herança: use apenas quando existir uma clara relação “é um” (ex.: um Circulo é uma Forma).
 - Agregação/Composição: prefira quando o relacionamento for “tem um” ou “é composto de” (ex.: uma Pessoa tem um Endereco).
 - Não force hierarquias de herança quando composição resolver melhor a modelagem.

Boas Práticas de Herança (4/4)

- Dicas de Engenharia de Software:
 - Projete superclasses para serem genéricas e reutilizáveis.
 - Evite criar hierarquias de herança muito profundas (preferir 2 ou 3 níveis).
 - Considere o impacto de mudanças futuras na superclasse sobre as subclasses existentes.
 - Sempre documente bem o comportamento esperado dos métodos que podem ser sobrescritos.

Recapitulando Herança em Java (1/2)

- **Herança** é um dos pilares da POO e permite reutilizar atributos e métodos de uma classe base em novas classes.
- **Superclasse**: classe mais genérica que define atributos e comportamentos comuns.
- **Subclasse**: especialização da superclasse, que herda e pode estender ou sobrescrever seus comportamentos.
- O Java utiliza a palavra-chave `extends` para declarar herança entre classes.
 - Não existe herança múltipla em Java.

Recapitulando Herança em Java (2/2)

- Subclasses herdam:
 - Métodos `public` e `protected`.
- Subclasses NÃO herdam:
 - Construtores.
 - Atributos `private`.
- A chamada ao construtor da superclasse deve ser feita com `super()`, sempre como **primeira linha** do construtor da subclasse.
- Toda classe em Java herda, direta ou indiretamente, da classe `Object`.

Exercícios propostos

Verificar atividade no Moodle